

Universidad Nacional Autónoma de México

Facultad de Ciencias

Reporte de uso de algún modelo de lenguaje

Equipo:

Aguilar Rosas Carlos

Cortes Leon Luis Gustavo

Trejo Garcia Ozkar Mauricio



- **Introducción**

Durante el desarrollo del proyecto se utilizan la herramienta de Inteligencia Artificial, conocida como ChatGPT, como apoyo para la resolución de dudas relacionadas con programación y procesamiento de imágenes

Las decisiones relacionadas con el diseño e implementación del proyecto fueron realizadas por los integrantes, y la IA se utilizó como herramienta de apoyo, no como sustituto del trabajo realizado.

- **Generar las figuras**

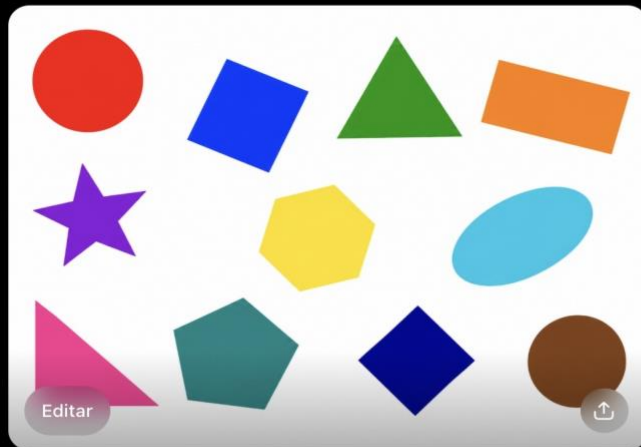
Se uso el siguiente prompt para generar las imagenes

“Genera una imagen con las siguientes características 1. El fondo de un solo color, uniforme (blanco de preferencia) 2. Cada figura tiene un color sólido, distinto entre figuras y distinto entre fondo 3. NO hay anti-aliasing ni colores intermedios en los bordes. 4. Las figuras no se traslapan entre si. 5. Tamaño, rotación y posición de las figuras son arbitrarios. 6 Para que detecte su color generalos dentro de la escala HSV”

Como al ver que esas imágenes no nos ayudarían mejor hicimos un script en python que generara dichas imágenes de acuerdo a la escala HSV

Posteriormente, las imágenes generadas fueron utilizadas como datos de prueba para verificar el funcionamiento del programa de reconocimiento de figuras. Se revisaron las características de las imágenes y se realizaron modificaciones o nuevas generaciones cuando estas no cumplían con las condiciones necesarias para las pruebas.

Genera una imagen con las siguientes características 1. El fondo de un solo color, uniforme (blanco de preferencia) 2. Cada figura tiene un color sólido, distinto entre figuras y distinto entre fondo 3. NO hay anti-aliasing ni colores intermedios en los bordes. 4. Las figuras no se traslapan entre sí. 5. Tamaño, rotación y posición de las figuras son arbitrarios. 6. Para que detecte su color generalízalos dentro de la escala HSV



Con tus valores:

Compartir

Figura	HSV	Color aproximado
Círculo	(0, 255, 255)	Rojo
Cuadrado	(15, 255, 255)	Naranja
Rectángulo	(30, 255, 255)	Amarillo
Rombo	(45, 255, 255)	Amarillo-verde
Trapezio	(75, 255, 255)	Verde
Triángulo isósceles	(105, 255, 255)	Azul-cian
Triángulo escaleno	(125, 255, 255)	Azul
Pentágono	(145, 255, 255)	Violeta
Hexágono	(160, 255, 255)	Magenta
Octágono	(90, 255, 180)	Verde/cian oscuro

Así que el pusimos el siguiente prompt:

“Genera un script en python con las 10 figuras”

Este script genera una imagen con 10 figuras, todas con:

- Fondo blanco uniforme.
- Un color sólido por figura.
- Colores distintos entre figuras.
- Sin degradados.
- Sin anti-aliasing.
- Sin traslapes.
- Tamaño, posición y rotación diferentes.
- Colores definidos inicialmente en HSV.
- Las figuras quedan listas para probar con `cv.inRange()`.

</> Python



Ejecutar

```
# Colores en HSV de OpenCV
# H: 0-179
# S: 0-255
# V: 0-255
COLORES_HSV = {
    "rojo": (0, 230, 230),
    "naranja": (15, 230, 230),
    "amarillo": (30, 230, 230),
    "verde": (60, 230, 230),
    "verde_azulado": (85, 230, 230),
```



</> Python



Ejecutar

```
def dibujar_figura(imagen, tipo, centro, tamaño, angulo, color):
    """
    Dibuja una figura utilizando colores sólidos.
    LINE_8 evita anti-aliasing.
    """

    cx, cy = centro

    if tipo == "circulo":

        cv.circle(
            imagen,
            (cx, cy),
            tamaño,
            color,
            -1,
            lineType=cv.LINE_8
        )

    elif tipo == "cuadrado":
```

El cual revisando, analizando en equipo se llegó a la conclusión de que no serviría todo el script y se volvió a hacer otro, tomando algunas ideas del que nos dio dicho prompt.

```
< img_hsv(1, 1) = 18, 8, 200] Principal Inbox Explorar Copilot ...

color = fig["color"]
f_type = fig["type"]

# 2. Dibujar la figura correspondiente sin anti-aliasing (LINE_4 o LINE_8)
if f_type == "circle":
    cv2.circle(img_hsv, (200, 200), 120, color, -1, lineType=cv2.LINE_4)

elif f_type == "square":
    pts = np.array([[100, 100], [300, 100], [300, 300], [100, 300]])
    cv2.fillPoly(img_hsv, [pts], color, lineType=cv2.LINE_4)

elif f_type == "rectangle":
    pts = np.array([[60, 140], [340, 140], [340, 260], [60, 260]])
    cv2.fillPoly(img_hsv, [pts], color, lineType=cv2.LINE_4)

elif f_type == "rhombus":
    pts = np.array([[200, 60], [340, 200], [200, 340], [60, 200]])
    cv2.fillPoly(img_hsv, [pts], color, lineType=cv2.LINE_4)

elif f_type == "trapezoid":
    pts = np.array([[120, 100], [280, 100], [340, 300], [60, 300]])
    cv2.fillPoly(img_hsv, [pts], color, lineType=cv2.LINE_4)

elif f_type == "tri_iso":
    pts = np.array([[200, 70], [330, 330], [70, 330]])
    cv2.fillPoly(img_hsv, [pts], color, lineType=cv2.LINE_4)

elif f_type == "tri_esc":
    pts = np.array([[100, 80], [350, 280], [120, 340]])
    cv2.fillPoly(img_hsv, [pts], color, lineType=cv2.LINE_4)

elif f_type == "pentagon":
    pts = np.array([[200, 60], [330, 160], [280, 330], [120, 330], [70, 160]])
    cv2.fillPoly(img_hsv, [pts], color, lineType=cv2.LINE_4)

elif f_type == "hexagon":
    pts = np.array([[130, 70], [270, 70], [340, 200], [270, 330], [130, 330], [60, 200]])
    cv2.fillPoly(img_hsv, [pts], color, lineType=cv2.LINE_4)

elif f_type == "octagon":
    pts = np.array([[140, 60], [260, 60], [340, 140], [340, 260], [260, 340], [140, 340], [60, 260], [60, 140]])
    cv2.fillPoly(img_hsv, [pts], color, lineType=cv2.LINE_4)

# 3. Convertir de HSV a BGR para guardar como BMP
img_bgr = cv2.cvtColor(img_hsv, cv2.COLOR_HSV2BGR)

# 4. Guardar archivo en formato BMP
file_path = os.path.join(output_dir, f"{fig['name']}.bmp")
cv2.imwrite(file_path, img_bgr)
```

- Funciones para poder usar

Se le pidió el siguiente prompt:

“Puedes darme un listado de funciones que se pueden usar para el proyecto y su breve descripción (pueden ser de python o OpenCv)”

Del cual solo se usaron en el proyecto

2. Preprocesamiento	
Estas son las más importantes para transformar la imagen antes de detectar figuras.	
Función	Descripción
<code>cv2.cvtColor()</code>	Convierte una imagen entre espacios de color, por ejemplo BGR → HSV o escala de grises.
<code>cv2.inRange()</code>	Crea una máscara seleccionando píxeles dentro de un rango.
<code>cv2.threshold()</code>	Convierte una imagen en una imagen binaria utilizando un umbral.
<code>cv2.adaptiveThreshold()</code>	Realiza una binarización adaptativa.
<code>cv2.bitwise_and()</code>	Realiza una operación AND entre imágenes/máscaras.
<code>cv2.bitwise_not()</code>	Invierte una imagen o máscara.
<code>cv2.bitwise_or()</code>	Realiza una operación OR.
<code>cv2.medianBlur()</code>	Reduce ruido utilizando un filtro de mediana.
<code>cv2.GaussianBlur()</code>	Suaviza la imagen utilizando un filtro gaussiano.
<code>cv2.morphologyEx()</code>	Realiza operaciones morfológicas.
<code>cv2.erode()</code>	Reduce regiones blancas de una máscara.
<code>cv2.dilate()</code>	Expande regiones blancas de una máscara.
<code>cv2.getStructuringElement()</code>	Crea elemento estructurante para operaciones morfológicas.

4. Análisis de contornos

Estas funciones son especialmente útiles para `classifier.py`.

Función	Descripción
<code>cv2.findContours()</code>	Obtiene el contorno de una figura.
<code>cv2.approxPolyDP()</code>	Aproxima un contorno utilizando menos puntos.
<code>cv2.arcLength()</code>	Obtiene el perímetro.
<code>cv2.contourArea()</code>	Obtiene el área.
<code>cv2.convexHull()</code>	Calcula la envolvente convexa.
<code>cv2.isContourConvex()</code>	Indica si un contorno es convexo.
<code>cv2.boundingRect()</code>	Obtiene el rectángulo delimitador.
<code>cv2.minAreaRect()</code>	Obtiene un rectángulo delimitador rotado.
<code>cv2.fitEllipse()</code>	Ajusta una elipse a un conjunto de puntos.

6. Funciones para analizar círculos

Estas pueden ser útiles para distinguir `0` de otras figuras.

Función/operación	Descripción
<code>cv2.contourArea()</code>	Área de la figura.
<code>cv2.arcLength()</code>	Perímetro de la figura.
<code>cv2.minEnclosingCircle()</code>	Círculo que contiene el contorno.
<code>cv2.boundingRect()</code>	Permite comparar ancho y alto.
<code>math.pi</code>	Se utiliza para calcular circularidad.

3. Detección de figuras

Esta es la parte central de `detector.py`.

Función	Descripción
<code>cv2.connectedComponents()</code>	Encuentra regiones conectadas en una imagen binaria.
<code>cv2.connectedComponentsWithStats()</code>	Encuentra regiones conectadas y además proporciona área, posición y tamaño.
<code>cv2.findContours()</code>	Encuentra los contornos de objetos en una imagen binaria.
<code>cv2.drawContours()</code>	Dibuja contornos sobre una imagen.
<code>cv2.boundingRect()</code>	Obtiene el rectángulo que contiene un contorno.
<code>cv2.contourArea()</code>	Calcula el área de un contorno.
<code>cv2.arcLength()</code>	Calcula el perímetro de un contorno.
<code>cv2.moments()</code>	Calcula momentos geométricos de un contorno.
<code>cv2.minEnclosingCircle()</code>	Encuentra el círculo mínimo que contiene un contorno.
<code>cv2.minAreaRect()</code>	Encuentra el rectángulo de área mínima que contiene un contorno.
<code>cv2.boxPoints()</code>	Obtiene los vértices de un rectángulo rotado.

- **Conclusión**

La Inteligencia Artificial nos ayudo con la generación de imágenes y funciones que podríamos usar en el proyecto el cual todo se corrobora con el equipo y para poder planear una solución al proyecto pues se eligió solo lo necesario, Lo demas se discutió, organizo y se hizo en conjunto con el equipo sin que la IA nos hiciera todo el proyecto.